

Review Text Analytics

BOW Representation

Concept	Description
Unstructured Data Analytics	Transforming raw text (e.g., 'Hello world') into numerical formats for ML.
Tokenization	Splitting text into tokens (words or subunits). Basic step before any numerical transformation.
Bag of Words (BOW)	Converts text into a vector of word frequencies, ignoring order.
Term-Document Matrix	Matrix representing documents with term frequencies (or binary), useful for ML.
Normalization: Stemming	Reduces words to root forms by removing suffixes (e.g., 'running' → 'run').
Normalization: Lemmatization	Maps words to their base form using dictionaries (e.g., 'feet' → 'foot').
Normalization: Stop Words Removal	Eliminates common, low-information words (e.g., 'is', 'the').
Low Frequency Filtering	Removes rare words that might not help generalization.
Python Implementation	Code examples using NLTK and scikit-learn for tokenization, vectorization, and normalization.

Exemplar Questions

What does a term-document matrix primarily represent?

- A. Document lengths
- B. Semantic similarity
- C. Word frequencies across documents**
- D. POS tags

Which of the following techniques is used to reduce words to their base forms using dictionary mappings?

- A. Stemming
- B. Lemmatization**
- C. TF-IDF
- D. N-Gram

TFIDF, n-gram, & POS Tagging

Concept	Description
TF-IDF Weighting	Assigns weights to terms based on their frequency in a document and rarity in the corpus. Highlights unique, informative words.
N-Grams	Captures local word sequences (e.g., pairs or triplets) to preserve partial order and semantic nuances. Extends BOW to include token phrases; helps capture context like 'not good' vs. 'good'.
POS Tagging	Assigns part-of-speech to each word (e.g., noun, verb) to enrich features and preserve short-sentence-level partial order.
Dimensionality and Sparsity	Higher-order n-grams and POS tagging can expand feature space, leading to high-dimensional sparse matrices.
Parse Trees (Not required)	Full syntactic structure of sentences, offering deep order information but hard to integrate into classic ML models.
Python: TF-IDF & N-Grams	Example code using <code>TfidfVectorizer(ngram_range=(1,2), min_df=2)</code> from sklearn to compute weighted features.
Python: POS Tagging	Use NLTK's <code>pos_tag()</code> and integrate with <code>CountVectorizer</code> or <code>TfidfVectorizer</code> for feature extraction.

Exemplar Questions

In a corpus of 1000 documents, one document is randomly picked. The document contains a total of 25 terms and the term "unstructured" appears 3 times. Which one might be the correct value for the term's TF-IDF, if the term "unstructured" appears in approximately one-fifth of the total documents?

- A. $\text{Log}(5) / (3 * 1000)$
- B. **$3 * \text{Log}(5) / 25$**
- C. $25 * \text{Log}(5) / 1000$
- D. $3 * 25 / \text{Log}(5)$

How many bi-grams can be generated from given sentence: "Social media generates lots of unstructured data".

- A. 5
- B. **6**
- C. 7
- D. 8

Similarity and Clustering

Concept	Description
Similarity Measures	Metrics like cosine similarity, Jaccard, and Overlap coefficients to compare document vectors .
Python: Similarity Calculation	Use <code>CountVectorizer</code> and <code>sklearn.metrics</code> to compute similarity between documents.
Clustering	Grouping documents into clusters based on similarity (e.g., K-means, Agglomerative, Divisive).
K-Means Clustering	Partitions documents into k clusters using centroids; Need to know how K clustered are computed ; implemented with <code>KMeans</code> from <code>sklearn</code> .
Python: Clustering	Code examples using <code>KMeans</code> and <code>AgglomerativeClustering</code> to create and label clusters.
Topic Extractions using Clustering	Use the centroids' position vectors (in the same space as any document's vector) to describe the topic; the topic keywords comes from top terms with the highest value in the centroid vector, capturing the most frequent terms in a cluster.
LDA (Latent Dirichlet Allocation)	Probabilistic model to describe documents as mixtures of topics; A documents can have relevance to multiple topics, each topic is a distribution over words, just like the centroids of the clustering. A topic can also be described by top words with the highest value in the topic vector
Python: LDA Modeling	TFIDF- or TF-based input, implemented with <code>LatentDirichletAllocation</code> in <code>sklearn</code> (or <code>gensim</code> , not required)
Clustering vs. LDA	Conventional clustering assigns each doc to one cluster; LDA allows membership in multiple topics.

Exemplar Questions

Which similarity metric measures the angle between two document vectors?

- A. Jaccard
- B. Cosine**
- C. Euclidean
- D. Manhattan

How does LDA improve on traditional clustering methods like K-Means in analyzing a corpus of news articles?

- A. LDA uses Euclidean distances for clustering
- B. LDA allows overlapping topic membership per document**
- C. LDA enforces equal-sized clusters
- D. K-Means supports word co-occurrence probabilities

Classification, Sentiment Analysis & Logit Model

Concept	Description
Classification Task Setup	Assign documents to one or multiple predefined categories. The general framework that covers real-world examples including spam detection, sentiment tagging, and content labeling.
Sentiment Analysis Basics	Classify a document's emotional tone (e.g., positive/negative). Sentiment score is computed by combining vectorized word presence and terms' sentiment weights (weighted sum).
Python: Sentiment with VADER	Apply VADER's lexicon-based model to compute polarity scores
Learning a Classifier	Model learns mapping from input text features to labels using supervised data. Requires a dataset where true labels (e.g., sentiment) are known for training.
Logit Model & Weight Interpretation	Logistic regression uses a weighted sum of features ($X\beta$). β are the interpretable parameters showing each word's contribution to sentiment, which is equivalent to each word's sentiment score.
Training and Testing Split	Testing data simulates unseen cases to assess model generalization. Without it, accuracy might be inflated due to overfitting to training data.
How Parameters are Learned	Using labeled data, parameters β are optimized to maximize likelihood (log-likelihood) of observed labels. Continuously changing the value of β until that the accuracy is the highest.
Evaluation Metrics	Assess model performance using accuracy.
General Classification Models	Compare multiple models (Naive Bayes, SVM, Decision Trees) to identify best performance under same input features and labels.

Exemplar Questions

Why is a separate test set important in supervised learning?

- A. To speed up model training
- B. To evaluate generalization on unseen data**
- C. To reduce model size
- D. To ensure convergence

In a logit sentiment model, what does the vector β represent?

- A. The input text tokens
- B. The model output class
- C. The sentiment weights assigned to terms**
- D. The embedding layer

Other Classification Models

Concept	Description
Human vs Machine Classifiers	Transition from hard-coded rules to models that learn patterns from data (both representation and label).
Training a Model	Use labeled data $[X, c]$ to learn mappings. Split into training and test sets to ensure generalization and avoid overfitting.
Improving the Model	Refine representation model (e.g., n-grams, min_df) and classification model (try different models); Try to increase the sample size. Set a fair contests across different modeling (using the same training data and testing data).
Naive Bayes	Baseline model assuming conditional independence between features. Fast and simple but sometimes less accurate.
Support Vector Machine (SVM)	Finds maximum-margin hyperplane while minimizing the wrong classifications . Can be extended to non-linear cases using higher-order polynomials.
Decision Trees and Forests	Trees recursively split data; Random Forests average over many trees to reduce variance and using bagging to prevent overfitting.
Python Implementations	Trains and evaluates classifiers (Logit, SVM, NB, RF) using `sklearn`. Accuracy is used for comparison.
Evaluation and Overfitting	Use test accuracy. Overfitting occurs when accuracy in the testing data is worse than that in the training data, meaning the model learns noises from the training data

Exemplar Questions

If you observe that a Random Forest model trained on TF-IDF features performs exceptionally well on training data but poorly on validation data, which strategy would best mitigate overfitting without sacrificing much model flexibility?

- A. Increase the maximum depth of each tree
- B. Decrease the number of trees in the forest
- C. Increase randomness by subsampling both features and data points (bootstrap aggregating)**
- D. Switch from TF-IDF to raw counts

Which statement should I use to replace (???,???) in the code below to evaluate the performance of the model?

```
# training
NBmodel.fit(training_x, training_c)
y_pred_NB = NBmodel.predict(testing_x)
```

```
# evaluation
acc_NB = accuracy_score(???, ???)
print("Naive Bayes model Accuracy::
{:.2f}%".format(acc_NB*100))
```

- A. (training_c, y_pred_NB)
- B. (training_c, testing_x)
- C. (testing_x, testing_x)
- D. (testing_c, y_pred_NB)**

ANN & Image Classification

Concept	Description
Neural Network Basics	Artificial Neural Networks (ANNs) are inspired by biological neurons; they process information through connected layers with weights and activation functions.
Comparison to Classic Models	ANNs differ from models like logit and SVM by learning non-linear relationships through multiple hidden layers.
Layer Design and Parameters	Neural networks can be shallow or deep. Adding neurons or layers increases model capacity but also the number of parameters significantly. Calculate the (approximate) numbers of parameters.
Activation Functions	Functions like ReLU or sigmoid add non-linearity to the model, enabling richer feature learning.
Deep Learning and Feature Hierarchy	Deep neural networks learn multiple layers of features from low-level to high-level representations automatically. Most parameters are from the 1 st layer, whose number of parameters depends on high-the inputs size . High-dimensional inputs like images leads to too many parameters. The advantage of deep designs over shallow ones in terms of number of parameters needed.
Training Needs and Scalability	Larger networks require more data to generalize well.
Python: MLPClassifier	Use `MLPClassifier` from `sklearn.neural_network` to train deep learning models; how to specify the layers and neurons in MLPClassifier.
Image Representation and Classification	Convert RGB 2d images to vectors using 'flatten', resulting in 2d-matrix input. then apply any classic machine learning models on it, just like that on texts' vectorizations.

Exemplar Questions

You specified a DLmodel =
MLPClassifier(hidden_layer_sizes=(2,3))

What is its underlying architecture?

- A. 3 layers and 2 neurons in each
- B. 2 layers and 3 neurons in each
- C. 2 layers, first layer has 2 neurons, second has 3 neurons**
- D. 2 layers, first layer has 3 neurons, second has 2 neurons

You have an image that is saved as a jpg file. In each row, it has 100 pixels. In each column, it has 200 pixels. What is the #dimensions if you flattened the image?

- A. [200, 100, 3]
- B. [20000, 3]
- C. [60000, 1]**
- D. [20000, 1]

Full-Order Representation and Word Embedding

Concept	Description
Index-based Encoding	Assigns each word a unique index; Preserve the original order of terms in the document ; Simple but fails to capture semantic meaning or distance between words.
One-Hot Encoding	Categorical and binary version of index ; Each word is a binary vector ; Preserve the original order of terms in the document; Equivalent to BoW-style of every term; Can be converted to BOW (unordered one hot) by losing order info (but cannot be converted from BOW); High-dimensional and sparse representation.
Compare to Other Representations	Preserve more (and full) order information. N-gram preserves phrase-level local order information. POS-tagging preserves only order within short-sentences.
Limitations of Basic Encoding	Index and one-hot encodings don't preserve similarity; all word vectors are orthogonal (equally unrelated).
Word Embedding	Maps words to dense, low-dimensional vectors in which distances reflect semantic similarity.
Semantic Arithmetic	Word vectors enable operations like king - man + woman $\approx$ queen, showing semantic relationships.
Word2Vec	One example of Word Embedding
GloVe	Another example of Word Embedding
Pre-trained Embeddings	Google Word2Vec and Stanford GloVe offer embeddings trained on massive corpora, ideal for small or noisy datasets. They are essentially a dictionary of {term: [vector value]}.
Embedding Layer in Neural Networks	Transforms index-based input into trainable, dense embedding vectors (of parameters) during model training; common in deep learning architectures (e.g., LSTM, RNN).

Exemplar Questions

You have a collection of 123 documents. The length of the longest document is 50, whereas that of the shortest is 20. There are 4000 unique words in this collection. Using padding, which of the following can be the right dimensionality of the one-hot-embedded collection?

A. (123, 50, 4000)

B. (123, 20, 4000)

C. (123, 4000)

D. (123, 50)

E. (123, 20)

F. (4000, 20)

G. (4000, 50)

What is the main advantage of word embeddings over one-hot encoding?

A. Faster training

B. Lower vocabulary size

C. Semantic similarity is preserved

D. No need for tokenization

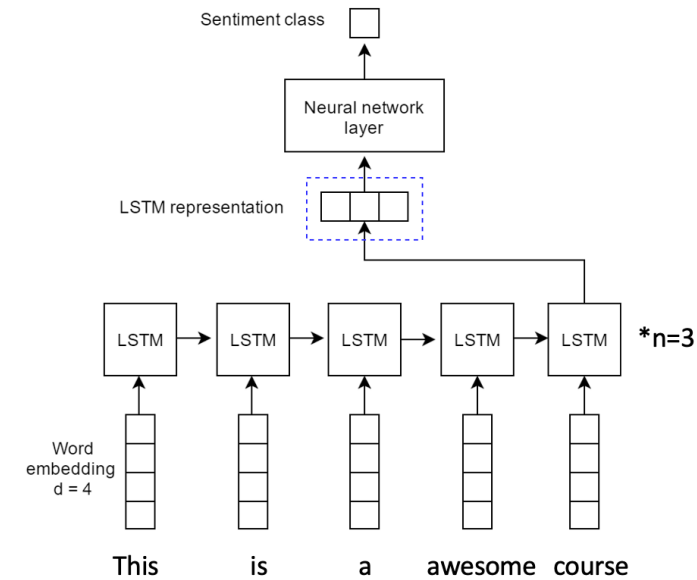
RNN and LSTM

Concept	Description
Importance of Order in Text	Order matters in tasks like machine translation, chatbots, and sentiment analysis; models must capture sequential dependencies based on representation that preserved order info (like index/one hot encoders)
Sequential Structures and RNNs	RNNs process input sequences one element at a time, maintaining hidden states that carry temporal dependencies. One RNN unit can handle the entire sequence, despite in multiple steps (each step one element). A step is not considered an RNN unit. By processing one element a time, the model can reduce the number of parameters needed because the number of parameters depends on the size of input (now only in each step)
Vanishing/Exploding Gradient Problem	During backpropagation in RNNs, gradients can shrink or grow exponentially, making long-term dependency learning difficult. Only learn from the later terms in a document.
LSTM Architecture	LSTM networks introduce memory cells and gating mechanisms (input, forget, output) to preserve long-term dependencies. Can learn from earlier terms in a document. Attention distribution.
Embedding Layer in LSTM	Maps sparse inputs (word indices) into dense embeddings, significantly reducing dimensionality and facilitating learning.
Python: LSTM for Sentiment Analysis	Training LSTM with Keras; includes embedding layer, LSTM unit, and dense output layer for binary classification.
Batch Size and Epochs	Batch size determines how many samples are processed before model weights are updated, it helps to reduce the size of object to handle a time; epoch is one full pass over the training data. The use of batches requires multiple epoch to ensure sufficient learning (e.g., earlier batches would have the opportunities to be learned after learning from the latter batches)
Sequence-to-Sequence Models	The learned order information can generate sequential classifications , generating outcomes as a sequence of terms, used for machine translation and chatbot. How to classify output.

Exemplar Questions

What problem does the LSTM architecture solve compared to vanilla RNNs?

- A. High dimensionality
- B. Slow training
- C. Vanishing gradient**
- D. Sparse input



How many LSTM layer(s) and LSTM unit(s) in each layer are in this architecture?

- A. LSTM layer and 3 LSTM units**
- B. LSTM layers and 5 LSTM units
- C. LSTM layers and 3 LSTM units
- D. LSTM layers and 3 LSTM units

Good luck!

